BASES DE DATOS II

Apuntes Completos para el Examen

Temas 6, 7 y 8

Silberschatz, Korth & Sudarshan — Database System Concepts, 6ª ed.

TEMA 6: ALMACENAMIENTO Y ESTRUCTURA DE ARCHIVOS

6.1 El modelo de almacenamiento en bases de datos

Los datos de una base de datos se almacenan permanentemente en almacenamiento no volátil (discos). El sistema operativo los organiza en archivos, y los SGBD los gestionan como secuencias de BLOQUES. El bloque es la unidad mínima de transferencia entre disco y memoria.

Concepto	Descripción	Tamaño típico
Bloque	Unidad de almacenamiento y transferencia E/S	4 KB a 8 KB
Registro	Una tupla/fila de la base de datos	Variable (depende del esquema)
Archivo	Colección de bloques que almacena una relación	Variable (GBs)

REGLAS FUNDAMENTALES de almacenamiento:

- Ningún registro ocupa más de un bloque (simplifica el acceso)
- Cada registro está completo en un único bloque (no se divide entre dos bloques)

6.2 Organización de Archivos (Registros de Longitud Fija)

6.2.1 Registros de longitud fija

Si todos los registros tienen el mismo tamaño, el almacenamiento es simple: se colocan uno tras otro. Cada bloque almacena tantos registros enteros como quepan (el resto del bloque queda sin usar para evitar que registros crucen fronteras de bloques).

PROBLEMA: Las eliminaciones dejan huecos. Soluciones:

Estrategia	Cómo funciona	Coste
Desplazamiento	Mover todos los registros posteriores hacia adelante para tapar el hueco	Alto: muchos accesos a disco
Sustitución	Mover el ÚLTIMO registro del archivo al hueco del registro borrado	Bajo: solo 2 accesos, pero el orden se pierde
Lista Libre (Free List)	La cabecera del archivo apunta al primer hueco. Ese hueco apunta al siguiente. Al insertar, se usa el primer hueco.	Muy bajo: O(1) para insertar y borrar

Lista Libre — la solución estándar

La cabecera del archivo guarda la dirección del primer registro borrado (hueco). Ese registro a su vez guarda la dirección del siguiente hueco, formando una 'lista enlazada' de espacios vacíos. Al insertar un registro nuevo: usar el primer hueco y actualizar la cabecera. Al borrar: añadir el hueco al inicio de la lista.

6.2.2 Registros de longitud variable

Ocurren cuando: los atributos tienen tipos VARCHAR, hay arrays o multiconjuntos, o se almacenan múltiples tipos de registros en un mismo archivo.

La representación interna de un registro de longitud variable tiene dos partes:

- Parte fija inicial: Para cada atributo de longitud variable, guarda un par (desplazamiento, longitud) que indica dónde empieza ese atributo y cuánto mide. Los atributos de longitud fija se almacenan directamente.
- Parte de datos variables: Los valores reales de los atributos variables, almacenados consecutivamente después de la parte fija.
- Mapa de bits de nulos (Null bitmap): Un bit por atributo que indica cuáles tienen valor NULL.

EJEMPLO
Registro de Instructor con ID (varchar 5), nombre (varchar 20), dept (varchar 20), sueldo (numeric).
Parte fija: [despl_ID=21, long_ID=5] [despl_nombre=26, long_nombre=10] [despl_dept=36, long_dept=10] [sueldo=65000]
Parte datos: '10101' 'Srinivasan' 'Informática'
El sistema accede al nombre con: inicio + desplazamiento = 21, longitud = 5

6.2.3 Estructura de Páginas con Ranuras (Slotted-Page)

Esta estructura es el estándar para almacenar registros de longitud variable dentro de un bloque. Organiza el bloque así:

Zona del bloque	Contenido	Crece hacia
Cabecera (inicio)	Número de entradas + puntero al final del espacio libre + array de (ubicación, tamaño) por registro	Hacia el final del bloque
Espacio libre (centro)	Espacio disponible para nuevos registros	—
Datos (final)	Registros almacenados consecutivamente	Hacia el inicio del bloque

```
Representacion visual de un bloque con slotted-page:

  +-------------------------------------------------------+
  | Cabecera:                                             |
  |  num_entradas = 3                                     |
```

```
  |  puntero_fin_libre = offset_X                            |
  |  entrada[1]: (ubicacion=900, tamano=80)   <- registro 1 |
  |  entrada[2]: (ubicacion=750, tamano=150)  <- registro 2 |
  |  entrada[3]: (ubicacion=600, tamano=150)  <- registro 3 |
  |                                                          |
  |               ... ESPACIO LIBRE ...                      |
  |                                                          |
  |  [registro 3: 150 bytes]  <- empieza en offset 600       |
  |  [registro 2: 150 bytes]  <- empieza en offset 750       |
  |  [registro 1:  80 bytes]  <- empieza en offset 900       |
  +----------------------------------------------------------+

  Al BORRAR registro 2:
  1. Su entrada en la cabecera se marca como borrada (tamano=-1)
  2. Los registros 3 y 1 se desplazan para que el espacio libre quede junto
  3. Las entradas de la cabecera se actualizan con las nuevas ubicaciones
  4. Los punteros EXTERNOS (indices, etc.) siguen apuntando a la ENTRADA DE
CABECERA,
     no al offset fisico del registro. Por eso el desplazamiento no invalida
punteros externos.
```

Ventaja clave: Los PUNTEROS EXTERNOS no apuntan directamente al registro, sino a su ENTRADA EN LA CABECERA. Así, cuando se elimina un registro y se compactan los datos (desplazando registros para que el espacio libre quede contiguo), solo hay que actualizar las entradas de la cabecera, no todos los punteros externos.

6.3 Objetos Grandes (BLOBs y CLOBs)

Cuando los datos (imágenes, vídeo, audio) superan el tamaño de un bloque, SQL usa tipos especiales:

Tipo	Nombre completo	Para qué
BLOB	Binary Large Object	Datos binarios: imágenes, vídeo, audio
CLOB	Character Large Object	Texto muy largo: documentos, libros

Estos objetos NO se almacenan en el registro normal. En su lugar, el registro contiene un puntero lógico al objeto, que se guarda en un archivo especial organizado mediante árboles B+ para permitir acceso eficiente a cualquier parte del objeto.

6.4 Organización de los Registros en Archivos

Hay cuatro estrategias principales para organizar los registros de una relación en archivos:

Organización	Cómo se almacenan	Mejor para	Peor para
Montículo (Heap File)	Sin orden específico, donde haya espacio	Inserciones rápidas, acceso secuencial completo	Búsquedas por valor específico

Organización	Cómo se almacenan	Mejor para	Peor para
Secuencial	Ordenados por una clave de búsqueda, con punteros entre registros	Recorridos ordenados, rangos	Inserciones y borrados frecuentes
Asociativo (Hash)	Función hash sobre atributo → determina el bloque	Búsquedas por igualdad exacta	Búsquedas por rango
Agrupación multitabla	Registros de varias tablas en el mismo bloque según relación JOIN	JOINs frecuentes entre esas tablas	Consultas sobre una sola tabla

6.4.1 Organización Secuencial en detalle

Los registros se ordenan por una CLAVE DE BÚSQUEDA (puede ser cualquier atributo, no necesariamente la clave primaria). Se usan punteros para mantener el orden lógico sin reordenar físicamente.

GESTIÓN DE INSERCIONES:

1. Localizar el registro antecesor según la clave
2. Si hay espacio libre en ese bloque (de borrados previos), insertar ahí
3. Si no hay espacio: usar un BLOQUE DE DESBORDAMIENTO (overflow block) y enlazar con punteros

Con el tiempo, los bloques de desbordamiento rompen la correspondencia entre orden físico y lógico, degradando el rendimiento. Hay que REORGANIZAR periódicamente el archivo (operación costosa, programada en horas de baja carga).

6.4.2 Organización en Agrupaciones de Varias Tablas

Esta técnica avanzada almacena registros de DIFERENTES TABLAS en el mismo bloque cuando están relacionados por un JOIN frecuente.

EJEMPLO PRÁCTICO
Si la consulta más frecuente es: SELECT * FROM departamento NATURAL JOIN instructor
Se almacenan juntos: registro del departamento 'Informática' + todos sus instructores.
Al leer el bloque del departamento, ya tienes todos sus instructores. Solo necesitas UNA lectura de bloque en lugar de N lecturas (una por instructor).

DESVENTAJA: Si necesitas consultar solo la tabla departamento, ahora cada bloque contiene pocos registros de departamento (la mayoría son de instructores), por lo que necesitas leer MÁS bloques para obtener todos los departamentos.

Solución: Cadenas de punteros que unen solo los registros de una misma tabla dentro del archivo agrupado.

6.5 Almacenamiento con Diccionario de Datos (Catálogo del Sistema)

El diccionario de datos (o catálogo del sistema) es la 'base de datos de la base de datos'. Almacena METADATOS: información sobre la estructura y el contenido de la BD.

Categoría de metadatos	Qué incluye
Esquema	Nombres de tablas y vistas, atributos de cada tabla, dominios y longitudes, restricciones de integridad
Vistas	Definición de cada vista (la consulta que la define)
Usuarios y permisos	Nombres de usuario, contraseñas, autorizaciones
Estadísticas	Número de tuplas por tabla, método de almacenamiento
Organización física	Tipo de organización (heap/secuencial/hash), ubicación en disco
Índices	Nombre, tabla, atributos, tipo de cada índice

EL PROBLEMA DE ARRANQUE (Bootstrap Problem)
Para acceder a cualquier tabla, el sistema consulta el diccionario de datos para saber dónde está esa tabla. Pero, ?dónde está el propio diccionario? La ubicación del diccionario debe estar 'hardcodeada' en el código del SGBD o en una ubicación fija del disco. Es el punto de entrada conocido que desencadena todo lo demás.

El diccionario de datos se almacena normalmente de forma DESNORMALIZADA (por ejemplo, la lista de atributos en una sola cadena) para maximizar la velocidad de acceso, aunque esto viola las formas normales.

6.6 Gestor de Memoria Intermedia (Buffer Manager)

6.6.1 ?Qué es la memoria intermedia?

La memoria intermedia (buffer) es la sección de la memoria RAM destinada a almacenar copias de bloques del disco. El GESTOR DE MEMORIA INTERMEDIA controla qué bloques están en memoria y cuándo se vuelcan al disco.

El objetivo es maximizar la probabilidad de que cuando se necesite un bloque, ya esté en memoria (evitar accesos a disco). Los accesos a disco son miles de veces más lentos que los accesos a RAM.

Proceso cuando se necesita un bloque:

4. Si el bloque YA está en el buffer: devolver la dirección en memoria (muy rápido)
5. Si NO está: buscar espacio libre en el buffer
6. Si no hay espacio libre: EXPULSAR un bloque (elegir cuál según la política de reemplazo)
7. Si el bloque expulsado fue MODIFICADO: volcarlo al disco primero
8. Leer el bloque solicitado del disco y colocarlo en el buffer

6.6.2 Diferencias con la memoria virtual del SO

El gestor de buffer de SGBD es más sofisticado que el gestor de memoria virtual del sistema operativo porque necesita características especiales:

Característica	?Por qué la necesita el SGBD?
Bloques clavados (Pinned)	Durante una actualización activa, el bloque NO puede escribirse al disco (violaria WAL). Se 'clava' en memoria.
Salida forzada (Forced Output)	En ciertos momentos (checkpoint, WAL), el sistema DEBE escribir un bloque al disco aunque no necesite ese espacio de buffer.
Políticas de reemplazo especializadas	El SGBD puede PREDECIR qué bloques se usarán próximamente mejor que el SO genérico.

6.6.3 Políticas de Reemplazo del Buffer

Cuando hay que expulsar un bloque, ?cuál elegir? La decisión impacta directamente en el número de accesos a disco:

Política	Cuándo usarla	Funcionamiento
LRU (Least Recently Used)	Caso general (lo que usa el SO)	Expulsar el bloque al que hace más tiempo que no se accede. Funciona bien para patrones de acceso con localidad temporal.
MRU (Most Recently Used)	JOINs por bucles anidados	Al procesar un JOIN $T1 \bowtie T2$ leyendo T2 entera por cada tupla de T1, el bloque de T2 usado MÁS RECIENTEMENTE es el que NO se usará hasta completar el ciclo. Expulsar el más reciente es lo óptimo.
Expulsión inmediata (Toss-immediate)	Acceso secuencial	Después de procesar la última tupla de un bloque en un recorrido secuencial, ese bloque ya no se volverá a usar. Liberarlo inmediatamente.

📌 PRIORIDADES DEL GESTOR DE BUFFER
El gestor siempre intenta NO expulsar: (1) el diccionario de datos y (2) los índices, porque se acceden con mucha frecuencia. Para ellos, LRU es ideal pues raramente envejecen.

6.7 Resumen del Tema 6

Concepto	Idea clave
Bloque	Unidad de E/S. Típicamente 4-8 KB. Registro debe caber completo.
Long. fija	Simple. Lista libre para gestionar borrados eficientemente.
Long. variable	Parte fija (offsets) + parte datos + null bitmap. Slotted-page en el bloque.
BLOB/CLOB	Objetos >1 bloque. Puntero en el registro, dato en árbol B+.
Heap	Sin orden. Rápido insertar, lento buscar.
Secuencial	Ordenado por clave. Bueno para rangos. Degradación con inserciones.
Agrupación multitabla	Tablas relacionadas en mismo bloque. Bueno para JOINs frecuentes.
Diccionario de datos	Metadatos de la BD. Almacenado como relaciones, desnormalizado para velocidad.
Buffer Manager	Gestiona RAM para bloques. Evita accesos a disco. LRU/MRU según patrón.
Bloques clavados	No escribibles al disco durante actualización (para WAL).
Salida forzada	Escritura obligatoria al disco en ciertos puntos (checkpoint).

TEMA 7: INDEXACIÓN Y ASOCIACIÓN (HASHING)

7.1 Extensiones de los Árboles B+

7.1.1 Organización de archivos con árbol B+ (los datos EN las hojas)

En el árbol B+ estándar, los nodos hoja contienen PUNTEROS a los registros en el archivo de datos. En la 'Organización de archivos con árbol B+', los nodos hoja CONTIENEN DIRECTAMENTE los registros (en lugar de punteros).

Tipo	Nodo hoja contiene	Ventaja	Desventaja
Índice B+ estándar	Claves + punteros a registros	Flexible, el archivo puede estar ordenado de otra forma	Dos accesos para obtener el registro (índice + archivo)
Organización de archivo B+	Los registros completos	Un solo acceso: el índice ES el archivo	Menor número de entradas por hoja (registros son más grandes que punteros)

REDISTRIBUCIÓN ENTRE HERMANOS: Para mejorar la utilización del espacio al insertar, en lugar de dividir siempre en dos nodos, se puede redistribuir entradas entre hasta m nodos hermanos. Esto garantiza que cada nodo tenga al menos $\lfloor (m-1)n/m \rfloor$ entradas (más llenos que el mínimo n/2 del B+ estándar).

7.1.2 Índices secundarios y reubicación de registros

Problema: Si la organización del archivo mueve registros de lugar (por ejemplo, en divisiones de nodos en organización B+), los índices secundarios que guardan punteros físicos a esos registros quedarían desactualizados.

Solución: En los índices secundarios, en lugar de punteros físicos, almacenar los VALORES DE LA CLAVE PRIMARIA del índice primario. La búsqueda requiere dos pasos:

9. Usar el índice secundario para encontrar el valor de la clave primaria
10. Usar el índice primario para encontrar el registro con esa clave primaria

Ventaja: Si un registro se mueve, el índice primario se actualiza, pero los índices secundarios (que guardan la clave primaria, no la ubicación) siguen siendo válidos.

7.1.3 Compresión de prefijos en índices sobre cadenas

Para índices sobre strings largos, los nodos internos del árbol B+ no necesitan almacenar la clave completa. Solo necesitan almacenar un PREFIJO lo suficientemente largo para distinguir los dos subárboles que separa.

EJEMPLO DE COMPRESIÓN DE PREFIJO
Claves: 'Silberschatz', 'Silver', 'Silas'

En un nodo interno entre 'Silas' y 'Silver', basta con almacenar 'Sile' o incluso 'Silf' en lugar de 'Silver' completo. Esto aumenta el fan-out del árbol (más entradas por nodo → árbol más bajo → menos accesos a disco).

7.1.4 Carga Masiva (Bulk Loading) de índices

Cuando hay que construir un índice para una tabla grande desde cero, insertar registro a registro es muy ineficiente (cada inserción puede requerir una lectura aleatoria del árbol).

El método eficiente es:

11. Crear un archivo temporal con todas las entradas del índice (clave + puntero al registro)
12. ORDENAR ese archivo por clave de búsqueda
13. Insertar las entradas ORDENADAS en el árbol, o mejor: construir el árbol de abajo hacia arriba

Al insertar entradas ya ordenadas, todas las entradas que van al mismo nodo hoja llegan juntas. Cada nodo hoja se escribe UNA sola vez (no hay que leerlo y re-escribirlo). La mejora de rendimiento puede ser de 1000x o más.

7.1.5 Árboles B vs Árboles B+

Característica	Árbol B	Árbol B+
Claves en nodos internos	SÍ: con puntero a dato asociado	SÍ: solo para navegación (sin dato)
Claves en nodos hoja	SÍ	SÍ: con todos los datos
Fan-out	MENOR (los punteros a datos ocupan espacio en internos)	MAYOR (solo claves de navegación en internos)
Recorrido secuencial	Difícil (nodos internos no están encadenados)	Fácil (hojas enlazadas)
Uso en la práctica	Poco usado	Estándar absoluto

📌 **NOTA IMPORTANTE**
En la literatura y en la industria, cuando alguien dice 'árbol B' para bases de datos, casi siempre se refiere al árbol B+. El árbol B (con datos en nodos internos) prácticamente no se usa en SGBD modernos.

7.1.6 Árboles B+ en Memorias Flash

Las memorias flash (SSD) son más rápidas para lectura (~1 microsegundo vs ~10 ms de un HDD), pero tienen la restricción de que no se pueden actualizar in-place: hay que borrar bloques enteros antes de reescribirlos (~1 ms para borrar).

Los árboles B+ se adaptan bien a flash: las búsquedas (solo lectura) se benefician enormemente de la velocidad de acceso aleatorio. Las actualizaciones son más lentas pero aceptables. El tamaño óptimo de nodo es más pequeño que para HDD (para adaptarse al tamaño de página de la flash).

7.2 Acceso con Múltiples Claves

Muchas consultas tienen condiciones sobre MAS DE UN atributo. Hay tres estrategias principales para manejar esto:

7.2.1 Usando varios índices de clave única — Interseccion de identificadores

Si tenemos un indice separado sobre dept_name y otro sobre salary, para la consulta WHERE dept_name='Finance' AND salary=80000 el algoritmo A9 hace esto:

14. Usar el indice dept_name → obtener lista de identificadores de registro (RIDs) de todos los instructores de Finance
15. Usar el indice salary → obtener lista de RIDs de todos los instructores con salary=80000
16. INTERSECTAR las dos listas de RIDs → quedan solo los RIDs que aparecen en AMBAS listas
17. Acceder a esos registros especificos del archivo de datos

Para la DISJUNCION (OR), el algoritmo A10 hace la UNION de las listas en lugar de la interseccion.

CUANDO LA INTERSECCION ES EFICIENTE Y CUANDO NO
EFICIENTE: Si ambas condiciones son muy selectivas (pocas filas cada una). La interseccion es rapida y solo se accede a pocos registros finales.

INEFICIENTE: Si alguna condicion devuelve una lista MUY GRANDE de RIDs, procesarla es costoso. Por ejemplo, si el 80% de instructores tiene salary < 80000, la lista del indice salary tiene 4000 RIDs de 5000 totales. Mejor usar solo el indice de dept_name (mas selectivo) y filtrar salary en memoria.

RETRO-ALTERNATIVA: Indice compuesto (dept_name, salary) si estas dos condiciones son frecuentes juntas.

7.2.2 Índices sobre varias claves (clave compuesta)

Un índice compuesto sobre (dept_name, salary) tiene las entradas ordenadas primero por dept_name y luego por salary dentro de cada dept. Esto permite:

Consulta	?Puede usar el índice compuesto?	Eficiencia
dept='Finance' AND salary=80000	SÍ, directamente	Alta
dept='Finance' AND salary < 80000	SÍ (igualdad en 1ª clave, rango en 2ª)	Alta
dept='Finance'	SÍ (igualdad en 1ª clave, todos los salary)	Alta
dept < 'Finance' AND salary < 80000	SÍ pero ineficiente (rango en ambas)	Baja
salary=80000 (solo 2ª clave)	NO directamente	Necesita escaneo secuencial

7.2.3 Índices de Cobertura

Un índice de cobertura almacena VALORES ADICIONALES de atributos (además de la clave y el puntero al registro). Si la consulta solo necesita esos atributos, puede responderse SIN acceder al archivo de datos.

EJEMPLO DE ÍNDICE DE COBERTURA
SELECT ID, salary FROM instructor WHERE ID = '12345'
Si el índice sobre ID también almacena el valor de salary junto al puntero:
→ La consulta se responde SOLO con el índice, sin acceder al archivo instructor.
Esto reduce accesos a disco a la mitad aproximadamente.

7.3 Asociación Estática (Static Hashing)

7.3.1 Funcionamiento básico

El hashing es una técnica que permite acceso DIRECTO a registros sin recorrer índices. Dada una clave de búsqueda K, una función hash h(K) determina inmediatamente en qué CAJÓN (bucket) está el registro.

Un cajón es un bloque de disco (o un conjunto de bloques) que almacena registros con el mismo valor hash. El número de cajones B es fijo en hashing estático.

EJEMPLO CONCRETO
Función: h(nombre_dept) = suma_caracteres(nombre_dept) mod 8
h('HISTORY') = 8+9+19+20+15+18+25 = 114, 114 mod 8 = 2 → cajón 2
Todos los instructores de History van al cajón 2.
h('PHYSICS') = ... → algún otro cajón
Para buscar instructores de History: calcula h('HISTORY')=2, accede solo al cajón 2.

7.3.2 Propiedades de una buena función hash

Propiedad	Descripción	Por qué importa
Uniformidad	Distribuye claves equitativamente entre cajones	Evita cajones llenos y cajones vacíos
Aleatoriedad	Claves similares van a cajones diferentes	Evita agrupaciones indeseadas
Determinismo	Siempre devuelve el mismo resultado para la misma clave	Necesario para encontrar los datos

Ejemplos de funciones hash sobre cadenas de caracteres tal como las muestra el profesor:

```
Funcion MODULO (la mas simple):
  h(k) = suma_de_representaciones_binarias_de_los_caracteres(k) mod B
  Ejemplo: h("HISTORY") = (72+73+83+84+79+82+89) mod 8 = 562 mod 8 = 2 -> cajon 2

  Funcion MULTIPLICACION POR CONSTANTE (mas uniforme):
  Si la clave es una cadena de n caracteres y c es una constante:
  h(k) = (s[0]*c^(n-1) + s[1]*c^(n-2) + ... + s[n-1]) mod B
  Donde s[i] es el valor ASCII del caracter i-esimo.
```

```
  Hash CRIPTOGRAFICO (MD5, CRC - para alta dispersion):
  MD5 (Message Digest 5): produce un hash de 128 bits
  CRC (Cyclic Redundancy Check): produce un hash de 32 o 64 bits
  Se usan cuando se necesita maxima uniformidad y resistencia a ataques.
  En bases de datos se usan principalmente para hashing extensible y verificacion
de integridad.
```

7.3.3 Desbordamiento y cadenas de overflow

Cuando un cajón se llena, se producen DESBORDAMIENTOS. La solución estándar es la CADENA DE DESBORDAMIENTO (overflow chaining):

- Cada cajón tiene un puntero a una lista de bloques de overflow
- Si el cajón principal está lleno, se añade un bloque de overflow y se encadena
- Las búsquedas recorren la cadena completa (cajón principal + todos sus overflow)

Para minimizar el desbordamiento al crear la estructura inicial, se recomienda: nB = (nr / fr) * (1 + d), donde d ≈ 0.2 es un factor de corrección que deja ~20% de espacio libre.

Hay dos tipos de asociacion segun como se gestiona el desbordamiento:

Tipo	Como maneja el desbordamiento	Preferencia en SGBD
ASOCIACION CERRADA (Closed Hashing / Chaining)	Cuando un cajon se llena, se enlaza con cajones de desbordamiento en cadena. Los cajones de desbordamiento son bloques adicionales del mismo espacio de almacenamiento.	PREFERIDA en SGBD: mantiene los registros agrupados cerca de su cajon original
ASOCIACION ABIERTA (Open Hashing / Open Addressing)	Cuando un registro no cabe en su cajon, se inserta en el SIGUIENTE cajon disponible (por ejemplo, el siguiente cajón en secuencia). No hay cajones de desbordamiento separados.	POCO USADA en SGBD: los registros pueden acabar muy lejos de su cajon original, dificultando las busquedas

POR QUE LOS SGBD USAN ASOCIACION CERRADA
En la asociacion abierta, al borrar un registro hay que marcar su posicion como 'borrado pero usado' para que las busquedas de otros registros no se interrumpan prematuramente. Esto complica la gestion. La asociacion cerrada con cadenas de desbordamiento es mas simple y predecible.

7.3.4 Limitaciones del hashing estático

Limitación	Detalle
No soporta rangos	Al no haber orden entre cajones, no se pueden hacer búsquedas BETWEEN, >, <

Limitación	Detalle
Degradación con el crecimiento	A medida que hay más registros, más desbordamientos, peor rendimiento
Reorganización costosa	Cambiar el número de cajones requiere recalcular el hash de TODOS los registros

7.3.5 Índices Hash (vs Organización Hash)

El hashing también se puede usar para crear ÍNDICES (no solo organizar archivos):

- Organización hash: Los REGISTROS están en los cajones directamente
- Índice hash: Los cajones contienen PARES (clave de búsqueda, puntero al registro). El archivo de datos es independiente.

Un índice hash secundario sobre el atributo ID de instructor: h(ID) = suma_dígitos mod 8. Al buscar el instructor con ID='22222': h('22222')=10 mod 8=2, ir al cajón 2, encontrar el puntero al registro.

7.4 Hashing Dinámico (Extendible Hashing)

?Por qué?

El hashing estático tiene problemas cuando la base de datos crece (o decrece). El hashing extensible (extendible hashing) soluciona esto modificando dinámicamente la función hash sin necesidad de reorganización global.

7.4.1 Estructura del hashing extensible

El hashing extensible usa:

- Una función hash h que produce b bits (normalmente b=32)
- Una TABLA DE DIRECCIONES de tamaño 2^i, donde i es la PROFUNDIDAD GLOBAL
- Cada cajón tiene su PROFUNDIDAD LOCAL ij (número de bits que usa ese cajón para distinguir sus entradas)

Si i > ij, múltiples entradas de la tabla apuntan al mismo cajón. Esto indica que el cajón comparte espacio con 2^(i-ij) entradas de la tabla.

EJEMPLO PASO A PASO
Inicialmente: i=0, una entrada en la tabla apuntando a un cajón vacío.
Insertar Biology (h=0010...): va al único cajón. Cajón = [Biology].
Insertar CompSci (h=1111...): también al único cajón. Cajón = [Biology, CompSci].
Insertar Finance (h=1010...): cajón lleno → DIVIDIR:
- i=0 = ij → duplicar tabla: i=1, tabla tiene 2 entradas.
- Entradas que empiezan por 0: Biology → cajón 1
- Entradas que empiezan por 1: CompSci, Finance → cajón 2
- Tabla: entrada 0 → cajón1, entrada 1 → cajón2.

7.4.2 Inserción con división de cajones

Al insertar en un cajón lleno:

CASO A: ij = i (el cajón solo lo apunta UNA entrada de la tabla)

18. Hay que DUPLICAR la tabla: i → i+1, la tabla tiene el doble de entradas
19. Dividir el cajón j en dos cajones: j y z
20. Las entradas del cajón j con el bit i+1 = 0 van al cajón j; las con bit = 1 van al cajón z
21. Reintentar la inserción

CASO B: ij < i (el cajón lo apuntan VARIAS entradas de la tabla)

22. NO es necesario duplicar la tabla
23. Dividir el cajón j en dos cajones: j y z
24. Actualizar las entradas de la tabla para repartirlas entre j y z
25. Reintentar la inserción

VENTAJA CLAVE DEL HASHING DINÁMICO
Solo se reorganiza UN cajón (el que se divide). El resto de cajones NO se toca. Esto es mucho más eficiente que el hashing estático donde hay que reorganizar todo.

7.4.3 Comparación Hashing Estático vs Dinámico

Aspecto	Estático	Dinámico (Extensible)
Crecimiento	Degradación progresiva	Se adapta dividiendo cajones
Reorganización	Global (muy costosa)	Local (solo el cajón afectado)
Espacio	Puede desperdiciar (muchos cajones vacíos)	Eficiente (cajones se crean cuando se necesitan)
Búsqueda (sin overflow)	1 acceso	2 accesos (tabla + cajón)
Complejidad	Simple	Algo más compleja

7.5 Comparación: Indexación Ordenada vs Hashing

Criterio	Árbol B+ (ordenado)	Hash
Búsqueda por igualdad (=)	O(log n)	O(1) promedio
Búsqueda por rango (BETWEEN, >, <)	Eficiente (recorrido secuencial de hojas)	Imposible (no hay orden entre cajones)
Recorrido ordenado	Eficiente (las hojas están enlazadas en orden)	No soportado
Inserciones y borrados	O(log n) + posibles divisiones/fusiones	O(1) + posibles overflows
Mejor para	Consultas variadas, rangos, ordenaciones	Búsquedas exactas muy frecuentes, archivos temporales

Criterio	Árbol B+ (ordenado)	Hash
Usado en	La inmensa mayoría de los SGBD	Algunos SGBD para índices de hash y datos temporales

REGLA PRÁCTICA
Si las consultas principales son de IGUALDAD EXACTA (=) y no necesitas rangos: hashing. Para todo lo demás: árbol B+. En la práctica, casi todos los SGBD siempre tienen un árbol B+ disponible. El hashing se usa principalmente para estructuras temporales durante el procesamiento de consultas.

EJERCICIO 1 del Profesor: Estrategias de Indexación

Relación instructor. Estrategia A: índices separados sobre dept_name y salary.
Estrategia B: índice compuesto (dept_name, salary).
1. Para SELECT * WHERE dept_name='Finance' AND salary=80000, ?cómo funciona A con intersección?
2. Para SELECT * WHERE dept_name='Finance' AND salary < 80000, ?por qué B es más eficiente?
3. Para SELECT * WHERE dept_name < 'Finance' AND salary < 80000, ?sigue siendo B eficiente?

SOLUCIÓN:
1. ESTRATEGIA A - Intersección:
Índice dept_name → lista de punteros de todos los instructores de Finance
Índice salary → lista de punteros de todos con salary=80000
INTERSECTAR ambas listas → punteros que aparecen en AMBAS = instructores Finance con 80k
INEFICIENCIA: Si hay 500 instructores de Finance y 200 con salary=80000, hay que procesar 700 punteros para encontrar quizás solo 3. Si los conjuntos son grandes, el coste es alto.

2. ESTRATEGIA B para dept='Finance' AND salary < 80000:
El índice B+ sobre (dept_name, salary) está ordenado primero por dept, luego por salary.
Buscar 'Finance': localizar directamente el rango (Finance,-∞) a (Finance,80000).
Solo se acceden exactamente los registros relevantes. Muy eficiente.

3. ESTRATEGIA B para dept < 'Finance' AND salary < 80000:
Para cada dept anterior a 'Finance', hay que buscar todos los salary < 80000.
Como el índice ordena por dept primero y salary dentro de cada dept,
los registros con salary<80000 de cada dept están dispersos en diferentes posiciones.
Puede requerir un acceso por cada dept, con lectura de muchos bloques. INEFICIENTE.

EJERCICIO 2 del Profesor: Hashing Dinámico (Extensible)

Estructura de hashing extensible, máximo 2 registros por cajón.
Insertar en orden: Biology (h=0010...), CompSci (h=1111...), Finance (h=1010...).
1. Muestra el estado tras cada inserción.
2. ?Qué pasa al insertar Finance si el cajón está lleno?

3. ?Cuándo se usarían cajones de overflow a pesar de la división?

✅ **SOLUCIÓN:**

1. ESTADO INICIAL: i=0, una entrada, cajón vacío.
   Insertar Biology (h=0010...): cajón=[Biology]. Tabla: {0→cajón1}. i=0.
   Insertar CompSci (h=1111...): cajón=[Biology, CompSci]. Tabla: {0→cajón1}. i=0.
   Insertar Finance (h=1010...): cajón LLENO → DIVIDIR (ij=0=i=0, caso A).

2. DIVISIÓN al insertar Finance (caso ij=i → duplicar tabla):
   i: 0→1. Tabla ahora tiene 2 entradas: {00...→cajón?, 10...→cajón?}
   Redistribuir cajón original según 1er bit del hash:
   - Biology (h=0010...) → 1er bit=0 → cajón 1
   - CompSci (h=1111...) → 1er bit=1 → cajón 2
   Insertar Finance (h=1010...) → 1er bit=1 → cajón 2 (junto a CompSci). OK.
   Tabla: {0→cajón1=[Biology], 1→cajón2=[CompSci, Finance]}.

3. CAJONES DE OVERFLOW: Si todos los registros tienen el MISMO valor de hash
   (por ejemplo, todos se llaman exactamente igual), la división no ayuda porque
   todos irian siempre al mismo cajon. En ese caso se usa overflow chaining.

7.6 Resumen del Tema 7

Concepto	Idea clave
B+ con datos en hojas	Registros directamente en hojas. Elimina una E/S de acceso.
Compresión de prefijos	Nodos internos almacenan solo el prefijo mínimo distinguidor. Mayor fan-out.
Bulk loading	Ordenar primero, insertar ordenado. 1000x más rápido que inserción individual.
Índice secundario con clave primaria	Guarda PK en lugar de puntero. Tolera reorganizaciones del archivo.
Índice de cobertura	Incluye atributos adicionales en el índice. Evita acceder al archivo.
Hashing estático	h(K) mod B = cajón. O(1) búsqueda. No soporta rangos. Degradación con crecimiento.
Hashing extensible	Tabla de profundidad global + cajones con profundidad local. Se adapta dinámicamente.
Árbol B+ vs Hash	B+: general, rangos, ordenación. Hash: igualdad exacta rápida, archivos temporales.

TEMA 8: PROCESAMIENTO Y OPTIMIZACIÓN DE CONSULTAS

8.1 Visión General del Procesamiento de Consultas

Cuando el usuario escribe una consulta SQL, el SGBD pasa por tres fases antes de devolver el resultado:

Fase	Qué hace	Resultado
1. Análisis y traducción	Verificar sintaxis, comprobar que las tablas existen, traducir SQL a álgebra relacional interna	Expresión en álgebra relacional
2. Optimización	Explorar planes alternativos (equivalentes pero más eficientes), estimar costes usando estadísticas, elegir el plan óptimo	Plan de evaluación anotado
3. Evaluación	Ejecutar el plan elegido usando los algoritmos específicos indicados	Resultado de la consulta

🎯 **MOTIVACIÓN DE LA OPTIMIZACIÓN**
Para la consulta SELECT sueldo FROM profesor WHERE sueldo < 75000, hay dos expresiones equivalentes:
1. σ(sueldo<75000)(Π(sueldo)(profesor)) → primero proyecta (descarta columnas), luego selecciona
2. Π(sueldo)(σ(sueldo<75000)(profesor)) → primero selecciona (descarta filas), luego proyecta
La opción 2 es MUCHO más eficiente: si hay 1000 profesores pero solo 100 con sueldo < 75000, la opción 2 procesa 100 filas en la proyección, la opción 1 procesa 1000.

8.2 Medidas del Coste de una Consulta

En bases de datos grandes, el cuello de botella es el acceso a disco (miles de veces más lento que la RAM). Por eso el coste se mide principalmente en términos de E/S de disco:

Coste ≈ Nb × tb + Ns × ts

Parámetro	Significado	Valor típico
Nb	Número de bloques transferidos (leídos o escritos)	—
tb	Tiempo de transferencia de un bloque	~0.1 ms por bloque de 4KB

Parámetro	Significado	Valor típico
Ns	Número de búsquedas de disco (seek)	—
ts	Tiempo de búsqueda + latencia rotacional	~4 ms promedio

📌 **SIMPLIFICACIÓN**
En la estimación de costes, generalmente se asume que los bloques no están en el buffer (peor caso conservador). Esto hace la estimación independiente del estado del buffer, más reproducible y comparable entre planes.

8.3 Estrategias para la Operación de Selección

La operación de selección σθ(r) tiene múltiples estrategias según los índices disponibles y el tipo de condición:

Algoritmo	Descripción	Coste estimado	Cuándo usar
A1: Búsqueda lineal	Recorrer todos los bloques, verificar cada tupla	ts + br×tb (br=bloques de r)	Siempre aplicable. Referencia base.
A2: Índice primario, igualdad en clave	Recorrer árbol B+, leer el único registro	(hi+1)×(ts+tb) (hi=altura árbol)	Clave única, condición =
A3: Índice primario, no-clave, igualdad	Recorrer árbol B+, leer b bloques consecutivos	hi×(ts+tb) + b×tb	Atributo no único, hay varios registros seguidos
A4: Índice secundario, igualdad	Recorrer árbol B+, luego n accesos al archivo	(hi+n)×(ts+tb), n=registros encontrados	Índice no agrupado. Puede ser PEOR que lineal si n es grande.
A5: Índice primario, comparación (<,>)	Buscar el límite en B+, escanear secuencialmente	Igual que A3	Condición de rango, índice agrupado
A6: Índice secundario, comparación	Escanear hojas del B+, acceder a cada registro	Igual que A4, puede ser costoso	Condición de rango, índice no agrupado. Generalmente evitar.

⚠ **TRAMPA FRECUENTE**
El índice secundario (A4, A6) no siempre es mejor que la búsqueda lineal. Si la condición selecciona muchas filas (baja selectividad), cada acceso al registro puede ser en un bloque diferente, resultando en UN ACCESO A DISCO POR FILA. Para 10.000 filas = 10.000 accesos aleatorios, mucho peor que leer los bloques secuencialmente.

Selecciones complejas

Algoritmo	Para qué	Cómo funciona
A7: Un índice + filtrado	Conjunción (AND) con un índice sobre uno de los términos	Usar el mejor índice para un término, filtrar el resto en memoria
A8: Índice compuesto	Conjunción con índice sobre múltiples atributos	Una sola pasada del índice cubre todos los términos del AND
A9: Intersección de IDs	Conjunción con varios índices	Obtener listas de IDs de cada índice, intersectarlas, acceder solo a esos registros
A10: Unión de IDs	Disyunción (OR)	Obtener listas de IDs de cada índice, unirlas, acceder a todos esos registros. Si algún término no tiene índice → búsqueda lineal

8.4 Optimización de Consultas — Introducción

La optimización elige entre múltiples planes equivalentes el de menor coste estimado. La diferencia entre un plan bueno y uno malo puede ser de ÓRDENES DE MAGNITUD (100x, 1000x más lento).

El proceso tiene tres niveles:

- Nivel de álgebra relacional: transformar la expresión en una equivalente pero más eficiente (aplicar reglas de equivalencia)
- Nivel de estrategia: elegir qué algoritmo específico usar para cada operación (qué tipo de join, qué índice)
- Nivel de ejecución: coordinar la ejecución de los operadores (pipelining vs materialización)

COMANDO EXPLAIN
La mayoría de los SGBD permiten ver el plan elegido:
• PostgreSQL: EXPLAIN <consulta>
• Oracle: EXPLAIN PLAN FOR <consulta>
• SQL Server: SET SHOWPLAN_TEXT ON
• DB2: comandos equivalentes
El coste mostrado está en unidades internas del optimizador, no en segundos.

8.5 Reglas de Equivalencia Relacional

El optimizador usa estas reglas para transformar la expresión en formas equivalentes más eficientes:

Regla	Notación	Significado práctico
Cascada de σ	$\sigma(\theta 1 \wedge \theta 2)(E) = \sigma\theta 1(\sigma\theta 2(E))$	Dividir selecciones conjuntivas en selecciones individuales (permite push-down)

Regla	Notación	Significado práctico
Conmutatividad de σ	σθ1(σθ2(E)) = σθ2(σθ1(E))	Reordenar selecciones para aplicar primero la más selectiva
Cascada de Π	ΠL1(ΠL2(...(ΠLn(E))...)) = ΠL1(E)	De múltiples proyecciones, solo importa la más restrictiva
σ combina con ×	σθ(E1×E2) = E1⋈θE2	Un producto cartesiano con condición = join. SIEMPRE preferir el join.
Conmutatividad de ⋈	E1⋈θE2 = E2⋈θE1	Los joins son conmutativos. Permite reordenar.
Asociatividad de ⋈	(E1⋈E2)⋈E3 = E1⋈(E2⋈E3)	Los joins son asociativos. Permite elegir el orden óptimo.
Push-down de σ	σθ(E1⋈E2) = σθ(E1)⋈E2 si θ solo usa atributos de E1	Aplicar la selección ANTES del join, reduciendo el tamaño de la entrada.
Push-down de Π	ΠL(E1⋈θE2) = ΠL1(E1)⋈θΠL2(E2)	Proyectar antes del join, eliminando columnas innecesarias.

LA OPTIMIZACIÓN MÁS IMPORTANTE

El push-down de selecciones (aplicar σ lo antes posible) es la optimización de mayor impacto.
Ejemplo: profesor ⋈ enseña filtrado por dept='Música'
MAL: (profesor ⋈ enseña) → σ(dept='Música') → puede procesar millones de combinaciones
BIEN: σ(dept='Música')(profesor) ⋈ enseña → solo los profesores de Música hacen el join
Si hay 1000 profesores y 10 son de Música, el join es 100x más pequeño.

8.6 Ordenación de Joins y Optimizador Basado en Coste

Para multiples joins (r1 JOIN r2 JOIN r3), el orden importa enormemente. El objetivo es minimizar el tamano de las relaciones intermedias.

EJEMPLO: Para encontrar nombres de profesores de Musica con el nombre de la asignatura:

```
MEJOR orden:
1. sigma(dept='Musica')(profesor)  -> pocos profesores de Musica (ej: 10)
2. (10 profesores) JOIN ensenia  -> pocas combinaciones (ej: 30 filas)
3. (30 filas) JOIN Pi(id_asig,nombre_asig)(asignatura)  -> resultado pequeno

PEOR orden:
1. ensenia JOIN Pi(id_asig,nombre_asig)(asignatura)  -> relacion gigante (miles
de filas)
2. sigma(dept='Musica')(profesor) JOIN (tabla gigante)  -> join costoso
```

EVITAR PRODUCTOS CARTESIANOS

Si el orden de joins crea un producto cartesiano (join sin atributos comunes): profesor JOIN asignatura sin condicion -> n_profesores x n_asignaturas filas.
Si hay 1000 profesores y 5000 asignaturas: 5.000.000 filas intermedias.
El optimizador detecta esto y evita el orden que lo produce.

El Optimizador Basado en Coste — Como funciona internamente

El OPTIMIZADOR BASADO EN COSTE explora el espacio de todos los planes de evaluacion equivalentes y selecciona el de menor coste estimado. Para una consulta con N tablas en JOIN:

- Hay O(N!) ordenes distintos de reunion posibles
- Pero usando programacion dinamica: una vez encontrado el mejor plan para un subconjunto de relaciones, se puede usar para reuniones posteriores
- La complejidad se reduce a O(3^N) — mucho mejor que O(N!) para N grande

```
Algoritmo HallarMejorPlan(S) - pseudocodigo del libro:

  procedure HallarMejorPlan(S):
    if mejorplan[S].coste != infinito:
      return mejorplan[S]   // ya calculado, usar resultado cacheado

    if S contiene solo 1 relacion:
      establecer mejorplan[S] segun la mejor forma de acceder a S
      (busqueda lineal, indice, etc.)
    else:
      for each subconjunto no vacio S1 de S tal que S1 != S:
        P1 = HallarMejorPlan(S1)
        P2 = HallarMejorPlan(S - S1)
        A = mejor algoritmo para juntar los resultados de P1 y P2
        coste = P1.coste + P2.coste + coste de A
        if coste < mejorplan[S].coste:
          mejorplan[S].coste = coste
          mejorplan[S].plan = "ejecutar P1.plan; ejecutar P2.plan; unir con A"

    return mejorplan[S]
```

Este algoritmo tiene complejidad O(3^N) porque cada subconjunto se calcula una sola vez y se cachea. Para N=10 tablas: 3^10 = 59049 planes evaluados. Para N=20: 3^20 = 3.5 billones (demasiado, hay que usar heuristicas).

Pipelining vs Materializacion

Hay dos formas de coordinar la ejecucion de operadores encadenados:

Estrategia	Como funciona	Ventaja	Cuando usarla
MATERIALIZACION (Materialización)	Cada operador termina completamente y escribe su resultado en disco. El siguiente operador lee ese resultado.	Simple, siempre funciona. El resultado intermedio queda guardado.	Cuando el resultado intermedio es grande y se usara varias veces

Estrategia	Como funciona	Ventaja	Cuando usarla
PIPELINING (Canalización o Tuberia)	Los operadores se ejecutan en paralelo: el primero produce tuplas que el segundo consume sin escribirlas en disco. Las tuplas fluyen de operador en operador como en una tuberia.	MUCHO mas rapido: elimina las escrituras/lecturas de resultados intermedios al disco	Siempre que sea posible (la mayoria de los casos)

```
Ejemplo visual de PIPELINING:

  [Leer tabla instructor]
    |
    v  (tuplas fluyen sin escribir en disco)
  [Filtrar dept='Physics']
    |
    v  (solo las tuplas que pasan el filtro)
  [Proyectar nombre, sueldo]
    |
    v  (resultado final al usuario)

  En pipelining: cuando un bloque de datos sale del primer operador,
  ya se procesa en el segundo operador INMEDIATAMENTE, sin esperar
  a que el primero termine toda su tarea.

  Analogia: una cadena de montaje en una fabrica vs
  enviar todos los componentes a un almacen y luego hacer el siguiente paso.
```

VISTAS MATERIALIZADAS: una vista materializada es una vista cuyo resultado se ha calculado previamente y almacenado fisicamente en disco. Cuando el optimizador detecta que una consulta puede responderse con una vista materializada, la usa directamente en lugar de calcular el join original. Son especialmente utiles para consultas analiticas repetitivas sobre datos que no cambian frecuentemente.

8.7 Estadísticas del Catálogo para la Estimación de Costes

Para estimar el coste de un plan, el optimizador usa estadísticas almacenadas en el catálogo del sistema:

Estadística	Notación	Descripción
Número de tuplas	nr	Total de filas en la relación r
Número de bloques	br	Número de bloques que ocupa r
Tamaño de tupla	lr	Tamaño en bytes de una fila
Factor de bloqueo	fr	Tuplas que caben en un bloque (fr = br/nr)
Valores distintos	V(A,r)	Número de valores distintos del atributo A en r

Las estadísticas NO se actualizan con cada operación (sería muy costoso). Se actualizan periódicamente con comandos como ANALYZE (PostgreSQL) o RUNSTATS (DB2).

8.7.1 Estimación del tamaño de la selección

Tipo de selección	Estimación del número de tuplas resultantes
$\sigma_{A=a}(r)$ (igualdad)	nr / V(A,r) — asumiendo distribución uniforme
$\sigma_{A\leq v}(r)$ (comparación)	0 si v < min(A,r); nr si v ≥ max(A,r); en otro caso: nr × (v - min(A,r)) / (max(A,r) - min(A,r))
Conjunción (AND $\theta_1 \wedge \theta_2$)	nr × (s1/nr) × (s2/nr) × ... = nr × ∏(si/nr) — asumiendo independencia
Disyunción (OR $\theta_1 \vee \theta_2$)	nr × (1 - ∏(1 - si/nr)) — complemento de 'ninguna cumple'
Negación ($\neg\theta$)	nr - s (donde s es la estimación de la condición positiva)

8.7.2 Estimación del tamaño del join

Caso	Estimación
R ∩ S = {} (sin atributos comunes)	nr × ns (producto cartesiano)
R ∩ S es clave de R	ns (o menos): cada tupla de S se une con a lo sumo una de R
Clave foránea de S referencia R	Exactamente ns
Caso general (sin clave)	min(nr×ns/V(A,r), nr×ns/V(A,s)) — tomar la estimación menor

EJEMPLO
estudiante ⋈ matricula por el atributo ID.
nr=5000, V(ID,estudiante)=5000 (clave de estudiante)
ns=10000, V(ID,matricula)=2500
Estimación opción 1: 5000×10000/2500 = 20000
Estimación opción 2: 5000×10000/5000 = 10000
Tomamos el menor: 10000 filas estimadas.

8.7.3 Histogramas para mejores estimaciones

La suposición de distribución uniforme (todos los valores con igual probabilidad) es a menudo incorrecta. Para mejorar, los SGBD almacenan HISTOGRAMAS que dividen el rango de valores en intervalos con su frecuencia:

- Histograma de equianchura: intervalos del mismo tamaño
- Histograma de equiprofundidad: intervalos con el mismo número de tuplas

Con histogramas se puede hacer una estimación separada para cada intervalo y sumarlas, obteniendo resultados mucho más precisos que con distribución uniforme.

8.8 Resumen del Tema 8

Antes del resumen, un ejercicio completo del tipo que pone el profesor:

EJERCICIO del Profesor: Comparar costes de planes para una consulta

Relacion instructor con: nr=5000, br=100, V(dept_name,instructor)=50, V(salary,instructor)=5000.
Consulta: SELECT * FROM instructor WHERE dept_name='Comp.Sci' AND salary > 70000
Opciones: (A) Busqueda lineal, (B) Indice primario sobre dept_name (altura B+=2),
(C) Indice secundario sobre salary (altura B+=2).
Calcular coste estimado de cada plan. ts=4ms, tb=0.1ms.

SOLUCIÓN:

PLAN A: Busqueda lineal (A1)
  Coste = ts + br*tb = 4 + 100*0.1 = 4 + 10 = 14 ms
  Lee TODOS los bloques secuencialmente. Filtra dept_name y salary en memoria.

PLAN B: Indice primario sobre dept_name (A3 - no es clave)
  Cuantos instructores de Comp.Sci hay? Estimacion: nr/V(dept_name,r) = 5000/50 = 100 tuplas
  Cuantos bloques contienen esas 100 tuplas? Como el indice es PRIMARIO (agrupado),
  las tuplas estan en bloques consecutivos: 100/50 = 2 bloques
  Coste = (hi)*(ts+tb) + b*tb = 2*(4+0.1) + 2*0.1 = 8.2 + 0.2 = 8.4 ms
  [hi=altura=2 para descender el arbol, b=2 bloques de datos a leer]
  Despues: filtrar salary>70000 en memoria entre las 100 tuplas. Muy barato.

PLAN C: Indice secundario sobre salary (A6 - comparacion, no-clave)
  Cuantos instructores tienen salary>70000? Estimacion simplificada: ~1/3 del rango
  Supongamos salary entre 50000 y 200000. Tuplas con salary>70000:
  = nr * (max - 70000)/(max - min) = 5000 * (200000-70000)/(200000-50000)
  = 5000 * 130000/150000 = ~4333 tuplas
  Cada tupla puede estar en un bloque DIFERENTE (indice secundario, no agrupado).
  En el peor caso: 4333 accesos aleatorios al disco.
  Coste = (hi + n)*(ts+tb) = (2 + 4333)*(4+0.1) = 4335*4.1 = ~17773 ms
  MUCHO peor que A1 (14ms) y B (8.4ms)!

CONCLUSION: El Plan B es el mejor (8.4 ms).
El Plan C parece que tiene un indice pero resulta ser el peor de todos.
El optimizador calcula estos costes y elige B automaticamente.

LECCION: Un indice secundario sobre un atributo con baja selectividad (muchos resultados)
es PEOR que la busqueda lineal. El optimizador lo sabe y lo evita.

Concepto	Idea clave
Fases del procesamiento	Analisis+traduccion → Optimizacion → Evaluacion
Coste de consulta	Nb*tb + Ns*ts. El E/S de disco domina.
A1 (busqueda lineal)	Universal, O(br) accesos. Base de comparacion.
A2-A6 (indices B+)	Mas rapidos si hay indice y selectividad alta (pocos resultados)
A4/A6 (indice secundario)	CUIDADO: puede ser peor que lineal si devuelve muchas filas
Push-down de selecciones	La optimizacion mas importante: filtrar antes de joinear
Ordenacion de joins	Joins con relaciones pequenas primero. Evitar productos cartesianos.
nr/V(A,r)	Estimacion estandar del numero de tuplas de una seleccion por igualdad
Estadisticas del catalogo	nr, br, lr, fr, V(A,r). Se actualizan periodicamente (ANALYZE)
Histogramas	Mejoran la estimacion del catalogo al capturar distribuciones no uniformes
EXPLAIN	Muestra el plan elegido por el optimizador. Util para diagnostico.